

Endpoint SIEM Engineering with Wazuh, Windows and Sysmon

Adeleke Theophilus

2026/ASA/15643

Advanced Security Operations and

Cloud Computing

18/07/2026

EXECUTIVE SUMMARY

This laboratory investigated the deployment and use of Wazuh and Sysmon for Windows endpoint monitoring, event collection and threat detection within a controlled virtual environment. The primary objective was to demonstrate how endpoint telemetry can be collected, analysed and correlated to identify behaviours commonly associated with cyber-attacks while distinguishing them from legitimate administrative activity.

The laboratory began with the preparation of a secure virtual environment consisting of a Windows endpoint monitored by a Wazuh agent and configured with Microsoft Sysmon. Following successful validation of the monitoring infrastructure, baseline endpoint telemetry was established to understand normal system behaviour before generating controlled security events. A supplied PowerShell script was then executed to safely simulate techniques frequently observed during cyber intrusions, including encoded PowerShell execution, Certutil encoding and decoding operations, and Windows Scheduled Task creation.

The generated telemetry was investigated using Sysmon Process Creation events and the Wazuh Dashboard. Individual events were analysed to identify executable images, command-line arguments, parent-child process relationships, cryptographic hash values, user context and execution timestamps. Event correlation techniques were subsequently applied to reconstruct the complete sequence of activities, enabling the behaviour of the simulated attack techniques to be understood within their operational context.

In addition to investigating endpoint activity, custom Wazuh detection rules were reviewed and validated to identify encoded PowerShell execution and CertUtil usage. The effectiveness of these behavioural detection rules was assessed, together with recommendations for tuning detections to minimise false positives while maintaining visibility of suspicious activity.

Finally, all temporary artefacts generated during the laboratory were successfully removed, the virtual environment was restored to a secure state, and recommendations were developed to improve endpoint monitoring, detection engineering and incident

response processes. Overall, the laboratory demonstrated the importance of comprehensive endpoint telemetry, behavioural analysis and event correlation in supporting effective Security Operations Centre (SOC) investigations and modern threat hunting activities.

Learning Objectives

- Import and safely network a prebuilt SIEM appliance.
- Onboard a Windows endpoint and confirm agent health.
- Install Sysmon with a controlled configuration and collect its Operational channel.
- Interpret Windows and Sysmon process events.
- Create and test custom Wazuh rules for encoded PowerShell and certutil use.
- Document pipeline health, alert evidence, tuning considerations and cleanup.

Tools Used

- Windows 11 Virtual Machine: Endpoint used to generate and investigate security events
- Wazuh Manager: Centralised collection and analysis of security events
- Wazuh Dashboard: Threat hunting and event investigation
- Sysmon: Endpoint monitoring and detailed Windows event collection
- Windows PowerShell: Execution of laboratory scripts and event generation
- Certutil: Safe file encoding and decoding simulation
- Schtasks.exe: Creation, execution and deletion of scheduled tasks
- Event Viewer: Verification of Sysmon events
- VirtualBox: Virtualisation platform hosting the Windows VM

Methodology

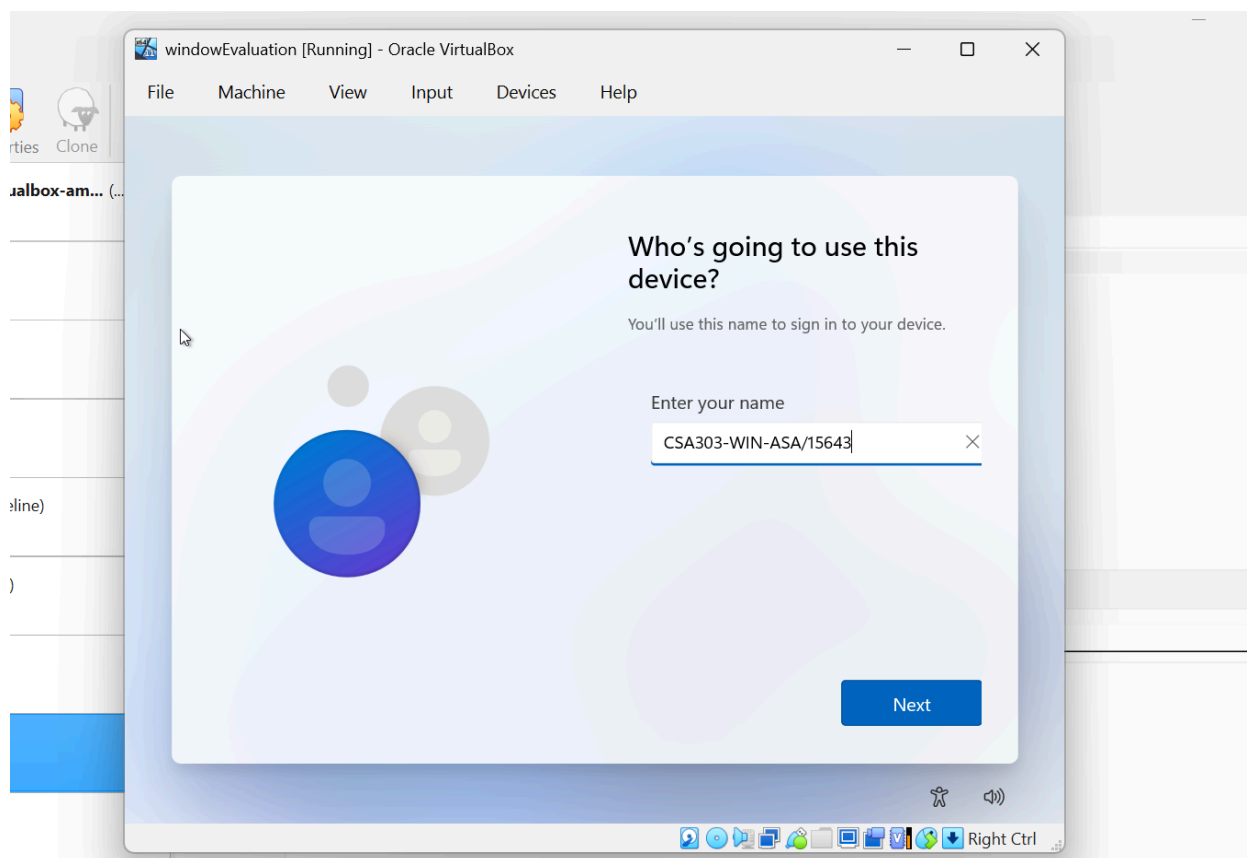
A controlled virtual laboratory environment was established to safely conduct endpoint monitoring, event generation, and threat investigation. The environment consisted of a Windows virtual machine configured with Microsoft Sysmon and the Wazuh agent, which communicated with the Wazuh Manager through an isolated Host-Only network. This configuration ensured that all generated telemetry remained within the laboratory while preventing unnecessary exposure to external networks.

The Windows virtual machine served as the monitored endpoint where all security events were generated. Sysmon was installed to provide enhanced Windows event logging, while the Wazuh agent collected these events and securely forwarded them to the Wazuh Manager. The Wazuh Dashboard was then used to perform threat hunting, event analysis and log correlation throughout the laboratory exercise.

```
PS C:\WINDOWS\system32> cd C:\Users\theop\Downloads
PS C:\Users\theop\Downloads> Get-FileHash .\wazuh-4.14.6.ova -Algorithm SHA512
>> Get-Content .\wazuh-4.14.6.ova.sha512
```

Algorithm	Hash	Path
-----	----	----
SHA512	40BC363DF468C8BAD64F283A6FE961ECC140FF6DB28F70DBB044DEA27E41330D5B7...	C:\Users\theop\Downloads\wazu...
	40bc363df468c8bad64f283a6fe961ecc140ff6db28f70dbb044dea27e41330d5b7c7279ead73d3695a569121a9a19f14d1bb1791e298d6db39cd20b03464d30	/tmp/wazuh-4.14.6-1.ova

OVA and checksum verify with SHA-512



Computer name set to was CSA303-WIN-ASA/15643

Both the Windows Evaluation and Wazuh were configured to communicate with each other. I set the IP addresses of Wazuh to 192.168.56.10 and 192.168.56.20 for Windows Evaluation under the Host-Only adapter. Still, Windows Evaluation was also exposed to the legitimate network so that it could reach the internet.

```
PS C:\WINDOWS\system32> ping 192.168.56.10

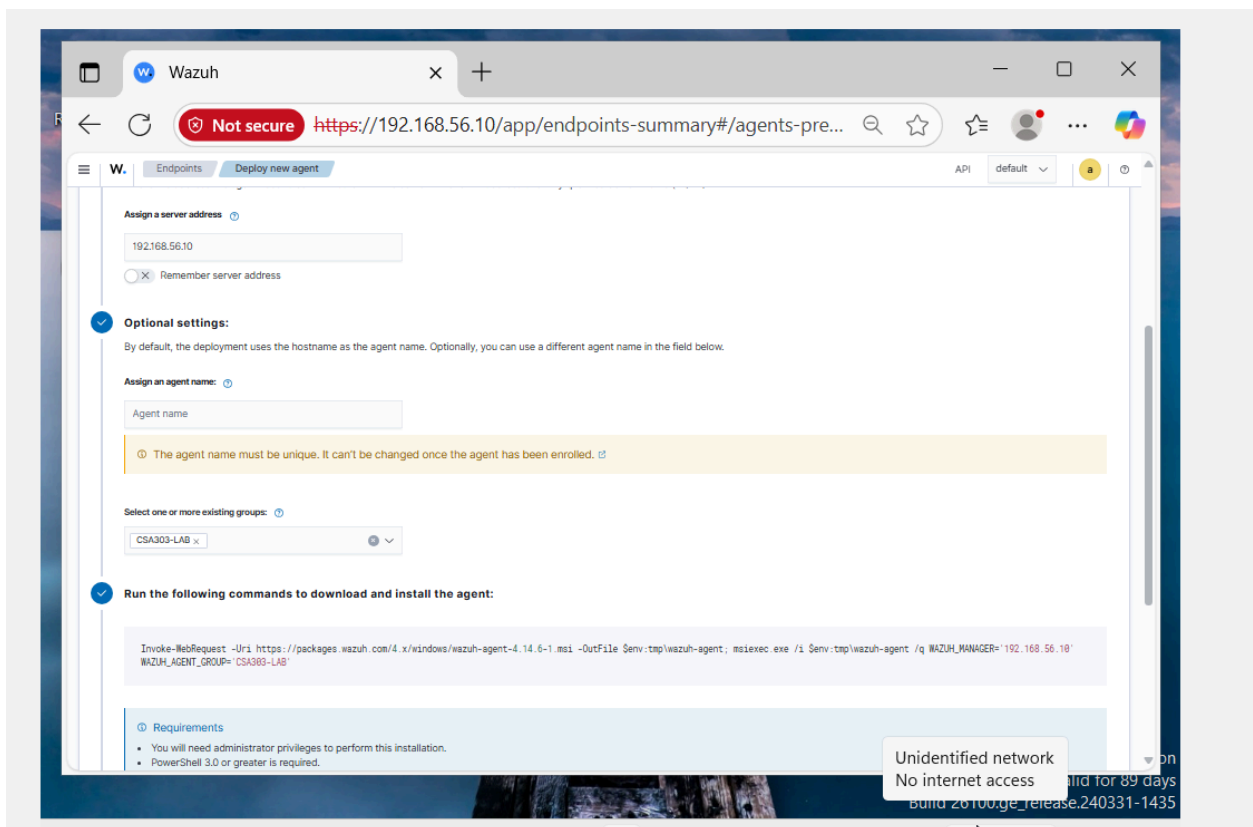
Pinging 192.168.56.10 with 32 bytes of data:
Reply from 192.168.56.10: bytes=32 time<1ms TTL=127
Reply from 192.168.56.10: bytes=32 time<1ms TTL=127
Reply from 192.168.56.10: bytes=32 time<1ms TTL=127
Reply from 192.168.56.10: bytes=32 time<1ms TTL=127

Ping statistics for 192.168.56.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms
PS C:\WINDOWS\system32> |
```

Onboard the Windows Agent

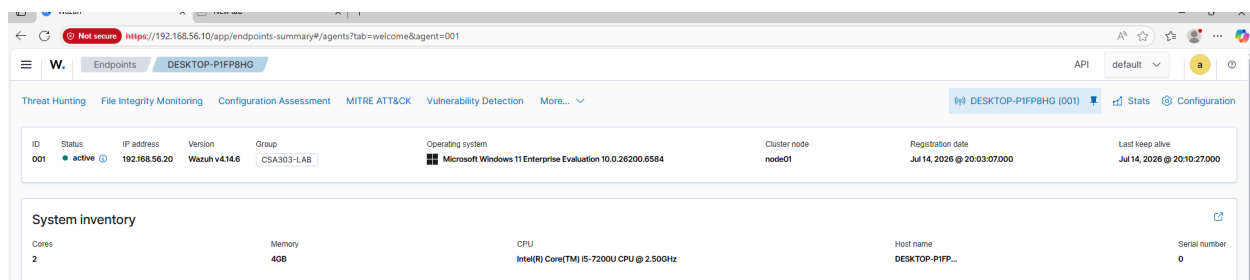
Before endpoint telemetry could be collected and analysed, the Windows virtual machine was required to be onboarded to the Wazuh Security Information and Event Management (SIEM) platform. Agent onboarding is a critical step because it establishes a trusted communication channel between the monitored endpoint and the Wazuh Manager, allowing security events generated on the endpoint to be securely transmitted for centralised monitoring and analysis.

The objective of this phase was to register the Windows endpoint with the Wazuh Manager, verify successful communication, and confirm that the endpoint was actively reporting security events before any investigation activities were performed.



```
PS C:\WINDOWS\system32> Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.6-1.msi -Out
File $env:tmp\wazuh-agent; msixec.exe /i $env:tmp\wazuh-agent /q WAZUH_MANAGER='192.168.56.10' WAZUH_AGENT_GROUP='
CSA303-LAB'
PS C:\WINDOWS\system32> NET START Wazuh
The Wazuh service is starting.
The Wazuh service was started successfully.
PS C:\WINDOWS\system32>
```

The generated installation command was copied from the Wazuh Dashboard and executed in an elevated PowerShell session using Administrator privileges. During execution, PowerShell initially blocked the installation because the current execution policy prevented the script from running. To resolve this issue, the following command was executed: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`



The screenshot shows the Wazuh Dashboard interface. The top navigation bar includes 'Threat Hunting', 'File Integrity Monitoring', 'Configuration Assessment', 'MITRE ATT&CK', 'Vulnerability Detection', and 'More...'. The main content area displays the agent status for 'DESKTOP-PIFP8HG (001)'. The agent is active, with IP address 192.168.56.20, version Wazuh v4.14.6, and group CSA303-LAB. The operating system is Microsoft Windows 11 Enterprise Evaluation 10.0.22000.6584. The cluster node is node01. The registration date is Jul 14, 2026 @ 20:03:07000, and the last keep alive is Jul 14, 2026 @ 20:10:27000. Below the agent status, the system inventory is shown, including 2 cores, 4GB memory, Intel(R) Core(TM) i5-7200U CPU @ 2.50GHz, host name DESKTOP-PIFP..., and serial number 0.

ID	Status	IP address	Version	Group	Operating system	Cluster node	Registration date	Last keep alive
001	active	192.168.56.20	Wazuh v4.14.6	CSA303-LAB	Microsoft Windows 11 Enterprise Evaluation 10.0.22000.6584	node01	Jul 14, 2026 @ 20:03:07000	Jul 14, 2026 @ 20:10:27000

System inventory				
Cores	Memory	CPU	Host name	Serial number
2	4GB	Intel(R) Core(TM) i5-7200U CPU @ 2.50GHz	DESKTOP-PIFP...	0

After installation, the Wazuh agent authenticated with the Wazuh Manager at 192.168.56.10 using a unique agent key and was assigned an Agent ID. The agent communicates securely with the manager over TCP port 1514 to send endpoint events, while TCP port 1515 is used during agent registration. The Active status and Last Keep-Alive timestamp shown in Figure 4.2 confirm that the endpoint is successfully connected and continuously forwarding security telemetry to the Wazuh Manager.

Install and Configure Sysmon

To enhance endpoint visibility beyond the standard Windows Event Logs, Microsoft Sysmon (System Monitor) was installed on the Windows virtual machine. Sysmon is a Sysinternals tool that continuously monitors system activity and generates detailed security events such as process creation, network connections, file creation, driver loading, and parent-child process relationships. These events provide security analysts with valuable forensic data for detecting suspicious behaviour and investigating potential security incidents.

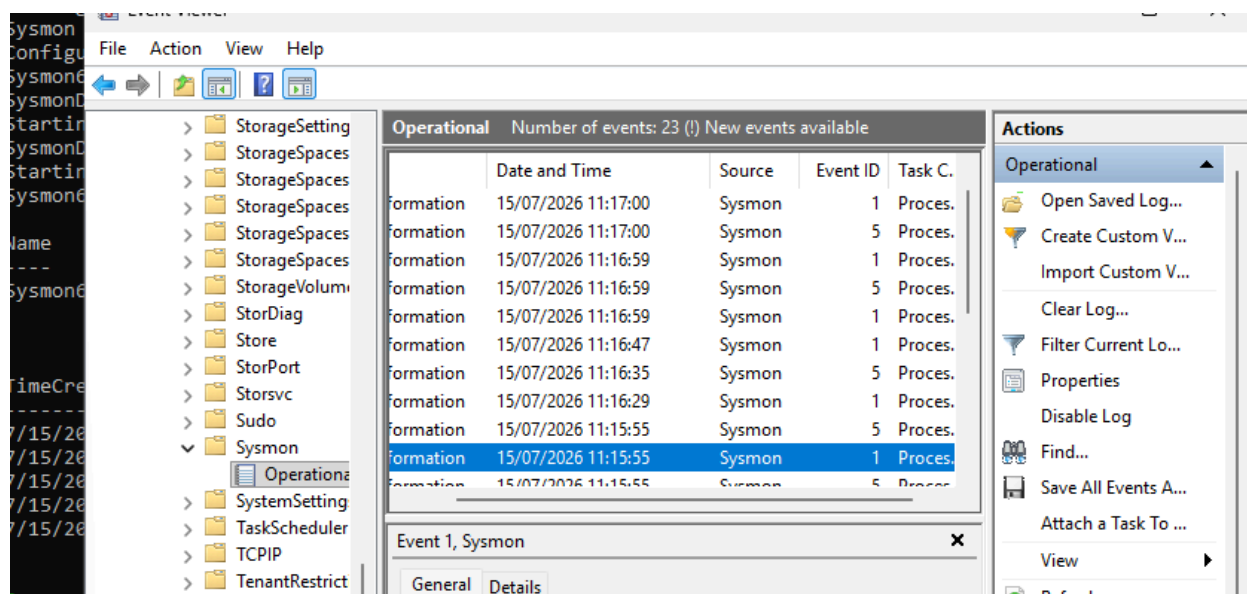
Sysmon Installation

Sysmon was downloaded from the official Microsoft Sysinternals website and extracted to C:\Tools\Sysmon. The supplied minimal-sysmonconfig.xml configuration file was copied into the same directory and reviewed before installation to understand the event filters being monitored.

The install-sysmon.ps1 script was then executed with Administrator privileges to install Sysmon and apply the configuration. During installation, the Sysinternals licence agreement was reviewed and accepted. After installation, the Sysmon service was verified to ensure it was running successfully.

```
TimeCreated      Id ProviderName
-----
7/15/2026 11:15:21 AM 5 Microsoft-Windows-Sysmon
7/15/2026 11:15:21 AM 1 Microsoft-Windows-Sysmon
7/15/2026 11:15:21 AM 1 Microsoft-Windows-Sysmon
7/15/2026 11:15:21 AM 4 Microsoft-Windows-Sysmon
7/15/2026 11:15:21 AM 16 Microsoft-Windows-Sysmon

Sysmon installation completed. Review the Operational log before continuing.
PS C:\Tools\Sysmon> _
```

Confirmation that Microsoft-Windows-Sysmon/Operational contains events.

After confirming that Sysmon was generating events locally, the `configure-wazuh-sysmon.ps1` script was executed with Administrator privileges. This script automatically backed up the existing `ossec.conf` configuration file and added the Microsoft-Windows-Sysmon/Operational event channel to the Wazuh agent configuration if it was not already present.

```
PS C:\Tools\Sysmon> .\configure-wazuh-sysmon.ps1
Added Sysmon Operational channel to Wazuh agent configuration.

Name      Status StartType
----      -
WazuhSvc  Running Automatic

Backup saved to C:\Program Files (x86)\ossec-agent\ossec.conf.20260715-112108.bak
PS C:\Tools\Sysmon>
```

Once the configuration was updated, the Wazuh service was restarted to apply the changes. This allowed Sysmon events to be forwarded from the Windows endpoint to the Wazuh Manager for centralised monitoring and analysis. The configuration backup created by the script was retained throughout the laboratory as a precaution, following good change management practices.

Event Viewer

Action View Help

StorageSetting
StorageSpaces
StorageSpaces
StorageSpaces
StorageSpaces
StorageVolum
StorDiag
Store
StorPort
Storsvc
Sudo
Sysmon
Operations
SystemSetting
TaskScheduler
TCPIP
TenantRestrict
TerminalService
TerminalService
TerminalService
TerminalService
TerminalService
TerminalService
Time-Service
Time-Service-I
Troubleshootin
TZSync
TZUtil
UAC
UAC-FileVirtua
UI-Search
UniversalTelen
USB-UCMUCS
USB-USBXHCI
User Control P
User Device Re
User Profile Se
User Loader

Operational Number of events: 126 (!) New events available

Level	Date and Time	Source	Event ID	Task C...
Information	15/07/2026 11:29:46	Sysmon	1	Proces...
Information	15/07/2026 11:29:46	Sysmon	5	Proces...
Information	15/07/2026 11:29:46	Sysmon	1	Proces...
Information	15/07/2026 11:29:46	Sysmon	5	Proces...
Information	15/07/2026 11:29:44	Sysmon	5	Proces...
Information	15/07/2026 11:29:43	Sysmon	1	Proces...
Information	15/07/2026 11:29:36	Sysmon	1	Proces...
Information	15/07/2026 11:29:36	Sysmon	5	Proces...
Information	15/07/2026 11:29:36	Sysmon	1	Proces...
Information	15/07/2026 11:29:36	Sysmon	5	Proces...
Information	15/07/2026 11:29:34	Sysmon	5	Proces...
Information	15/07/2026 11:29:34	Sysmon	1	Proces...
Information	15/07/2026 11:29:29	Sysmon	1	Proces...
Information	15/07/2026 11:29:29	Sysmon	1	Proces...
Information	15/07/2026 11:29:29	Sysmon	5	Proces...
Information	15/07/2026 11:29:29	Sysmon	5	Proces...
Information	15/07/2026 11:29:28	Sysmon	5	Proces...
Information	15/07/2026 11:29:27	Sysmon	1	Proces...
Information	15/07/2026 11:29:24	Sysmon	1	Proces...
Information	15/07/2026 11:29:20	Sysmon	1	Proces...

Event 1, Sysmon

General Details

Process Create:
RuleName: -
UtcTime: 2026-07-15 10:29:46.440
ProcessGuid: {73557c48-611a-6a57-d901-00000000b00}

Log Name: Microsoft-Windows-Sysmon/Operational
Source: Sysmon Logged: 15/07/2026 11:29:46
Event ID: 1 Task Category: Process Create (rule: ProcessCreate)
Level: Information Keywords:
User: SYSTEM Computer: DESKTOP-P1FP8HG

Actions

Operational

Open Saved Log...
Create Custom View...
Import Custom View...
Clear Log...
Filter Current Log...
Properties
Disable Log
Find...
Save All Events As...
Attach a Task To this ...
View
Refresh
Help
Event 1, Sysmon
Event Properties
Attach Task To This E...
Copy
Save Selected Events...
Refresh
Help

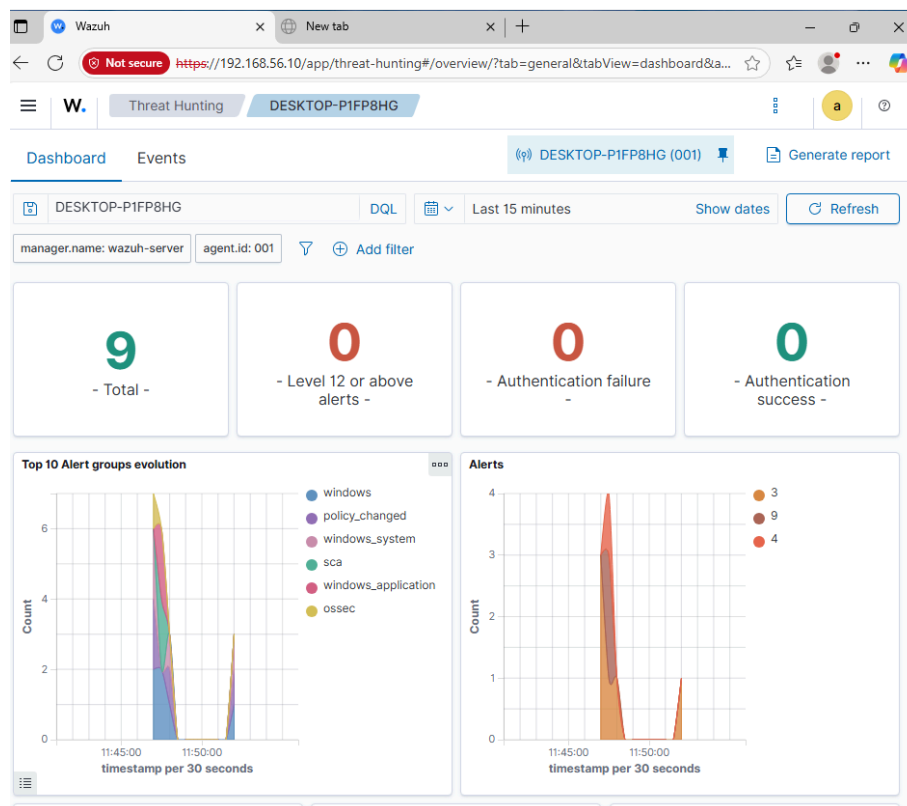
Restart the discovered Wazuh service and wait for new Sysmon events to appear

Baseline Telemetry

In this laboratory, baseline telemetry was established by analysing Sysmon Process Creation events collected by the Wazuh platform. These events provided visibility into the applications executed on the Windows endpoint, the relationships between parent and child processes, and the completeness of the collected telemetry. The information gathered during this stage formed the foundation for the later investigation of PowerShell, CertUtil and Scheduled Task activity.

To establish the baseline, the Threat Hunting module within the Wazuh Dashboard was used to retrieve recent Sysmon Process Creation events generated by the monitored endpoint. The search was restricted to the Windows virtual machine and the laboratory time window to ensure that only relevant events generated during the exercise were included in the analysis.

Restricting searches to a specific endpoint and time period(C1)



After retrieving the relevant Sysmon events, the recorded process images were reviewed to determine which applications executed most frequently during the observation period. Process frequency analysis provides an overview of normal endpoint behaviour and assists analysts in identifying unusual applications that may warrant further investigation.

The observed process frequency reflected the activities performed throughout the laboratory exercise, including PowerShell execution, Command Prompt processes, CertUtil operations and Windows Task Scheduler activity. These applications were expected because they were intentionally executed by the laboratory scripts to generate controlled security events.

Table of five most frequent images with counts (C2)

15 hits						
Jul 15, 2026 @ 12:10:56.650 - Jul 15, 2026 @ 13:10:56.651						
Export Formatted Reset view 719 available fields Columns Density 1 fields sorted Full screen						
	timestamp	agent.name	rule.desc...	r..	ru...	data.win.eventdata.image
	Jul 15, 2026 @ 13:05:18.517	DESKTOP-P1FP8HG	Powershell ...	4	9200	C:\\Windows\\System32\\cmd.exe
	Jul 15, 2026 @ 12:59:37.341	DESKTOP-P1FP8HG	Powershell ...	4	9200	C:\\Windows\\System32\\cmd.exe
	Jul 15, 2026 @ 12:56:21.583	DESKTOP-P1FP8HG	Application ...	12	9205	C:\\Windows\\System32\\sdbinst.exe
	Jul 15, 2026 @ 12:40:32.136	DESKTOP-P1FP8HG	Discovery a...	3	92031	C:\\Windows\\SysWOW64\\net1.exe
	Jul 15, 2026 @ 12:40:20.604	DESKTOP-P1FP8HG	Discovery a...	3	92031	C:\\Windows\\SysWOW64\\net.exe
	Jul 15, 2026 @ 12:39:50.148	DESKTOP-P1FP8HG	Discovery a...	3	92031	C:\\Windows\\SysWOW64\\net.exe
	Jul 15, 2026 @ 12:38:50.095	DESKTOP-P1FP8HG	Powershell ...	3	92021	C:\\Windows\\SysWOW64\\WindowsPowerSh...
	Jul 15, 2026 @ 12:38:20.242	DESKTOP-P1FP8HG	Powershell ...	3	92021	C:\\Windows\\SysWOW64\\WindowsPowerSh...
	Jul 15, 2026 @ 12:37:50.897	DESKTOP-P1FP8HG	Powershell ...	3	92021	C:\\Windows\\SysWOW64\\WindowsPowerSh...
	Jul 15, 2026 @ 12:37:18.266	DESKTOP-P1FP8HG	Powershell ...	3	92021	C:\\Windows\\SysWOW64\\WindowsPowerSh...
	Jul 15, 2026 @ 12:36:48.054	DESKTOP-P1FP8HG	Powershell ...	3	92021	C:\\Windows\\SysWOW64\\WindowsPowerSh...
	Jul 15, 2026 @ 12:36:18.011	DESKTOP-P1FP8HG	Powershell ...	3	92021	C:\\Windows\\SysWOW64\\WindowsPowerSh...
	Jul 15, 2026 @ 12:35:47.103	DESKTOP-P1FP8HG	Powershell ...	3	92021	C:\\Windows\\SysWOW64\\WindowsPowerSh...
	Jul 15, 2026 @ 12:35:15.683	DESKTOP-P1FP8HG	Powershell ...	3	92021	C:\\Windows\\SysWOW64\\WindowsPowerSh...
	Jul 15, 2026 @ 12:34:44.931	DESKTOP-P1FP8HG	Powershell ...	3	92021	C:\\Windows\\SysWOW64\\WindowsPowerSh...

Rows per page: 15

< 1 >

cmd.exe,sdbinst.exe,net1.exe,powershell.exe, certutil.exe

Parent–Child Process Relationship Analysis (C3)

Parent-child process relationships reveal the execution chain that led to a particular process, enabling analysts to reconstruct user activity and detect suspicious process hierarchies commonly associated with malware and attacker behaviour.

During the baseline analysis, the parent process associated with each significant process creation event was examined using the Sysmon ParentImage field. This information allowed the execution sequence to be reconstructed and verified against the expected behaviour of the laboratory exercise.

Parent Process: powershell.exe

11eName	
t	data.win.eventdata.parentCommandLine
	"C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe"

Child Process: cmd.exe

	C:\\Windows\\System32\\cmd.exe
--	--------------------------------

Short Interpretation: PowerShell launched the Windows Command Prompt to perform scripted operations

Parent Process:svchost.exe

t	data.win.eventdata.parentImage	C:\\Windows\\System32\\svchost.exe
---	--------------------------------	------------------------------------

Child Process: sdbinst.exe

C:\\Windows\\System32\\sdbinst.exe

Short Interpretation: svchost.exe launched sdbinst.exe to perform an application compatibility installation. This is legitimate Windows behaviour but should be verified if unexpected.

Telemetry Quality (C4)

Data Quality Observation

During the initial validation of endpoint telemetry, Sysmon was successfully generating events within the Microsoft-Windows-Sysmon/Operational event log; however, these events were not appearing in the Wazuh Dashboard. This indicated that the issue was not with Sysmon itself, but with the Wazuh agent configuration, which had not yet been configured to monitor the Sysmon Operational event channel.

To resolve this issue, the supplied configuration script `configure-wazuh-sysmon.ps1` was executed with Administrator privileges. The script automatically created a timestamped backup of the existing `ossec.conf` configuration file before modifying it. It then added the Microsoft-Windows-Sysmon/Operational event channel to the Wazuh agent configuration, enabling the agent to collect and forward Sysmon events to the Wazuh Manager.

GENERATE SAFE TEST EVENTS

A PowerShell script (generate-safe-events.ps1) was supplied to generate a sequence of benign security events. The script was specifically designed to demonstrate techniques that attackers frequently abuse, including encoded PowerShell execution, CertUtil file encoding and decoding, and Scheduled Task creation. Unlike real malware, the script was restricted to writing files within the C:\CSA303-Generated directory, creating a temporary scheduled task, and removing generated artefacts after execution. At no point did the script download external payloads, disable security software or establish persistent access to the endpoint.

The purpose of generating these events was to produce realistic telemetry that could later be analysed using Wazuh and Sysmon while maintaining a secure and isolated laboratory environment.

Before executing the laboratory script, the PowerShell execution policy was temporarily modified for the current session. This allowed the supplied script to execute without permanently changing the system's PowerShell security configuration. Limiting the policy change to the current PowerShell process is considered a safer administrative practice because the original execution policy is automatically restored when the session terminates.

```
PS C:\Users\CSA303-WIN-ASA15643\Desktop\CSA303LAB1> Set-ExecutionPolicy -Scope Process Bypass

Execution Policy Change
The execution policy helps protect you from scripts that you do not trust. Changing the execution policy might
expose you to the security risks described in the about_Execution_Policies help topic at
https://go.microsoft.com/fwlink/?LinkID=135170. Do you want to change the execution policy?
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): y
PS C:\Users\CSA303-WIN-ASA15643\Desktop\CSA303LAB1>
```

Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass

Following successful execution of the command, the laboratory script was launched using the student's allocated identifier

```
PS C:\Users\CSA303-WIN-ASA15643\Desktop\CSA303LAB1> .\generate-safe-events.ps1 -StudentId YOUR-STUDENT-ID
CSA303_START_UTC=2026-07-15T12:50:59.0127207Z
CSA303_END_UTC=2026-07-15T12:51:05.8009385Z
Evidence folder: C:\CSA303-Generated
All actions are local and benign. Run cleanup-safe-events.ps1 after evidence capture.
PS C:\Users\CSA303-WIN-ASA15643\Desktop\CSA303LAB1>
```

The script completed successfully and displayed the event generation process's UTC start and end timestamps. These timestamps were subsequently used to restrict searches within the Wazuh Dashboard, ensuring that only telemetry generated during the laboratory exercise was included in the investigation.

INCIDENT INVESTIGATION AND EVENT CORRELATION

Following the successful generation of the controlled security events, the investigation phase focused on analysing the telemetry collected by Sysmon and forwarded to the Wazuh platform. The objective was to identify the generated PowerShell, CertUtil, and Scheduled Task events, reconstruct the order in which they occurred, and determine whether the observed behaviour represented legitimate administrative activity or resembled malicious execution.

Rather than analysing individual events in isolation, the investigation emphasised event correlation. Correlation involves linking multiple independent events using common attributes such as timestamps, process identifiers, parent-child relationships and command-line arguments. This methodology enables security analysts to reconstruct the complete execution chain and determine how individual actions relate to one another.

The UTC timestamps generated by the PowerShell script were used to define the investigation window. Restricting the search to the known execution period reduced unrelated background activity and ensured that only events generated during the laboratory exercise were included in the analysis.

Investigation of Encoded PowerShell Execution

During the investigation, Sysmon Process Creation events were examined to identify PowerShell processes executed with the -EncodedCommand parameter. A matching event was successfully identified within the investigation time window.

The event contained detailed metadata including the executable image, complete command-line arguments, parent process information, process identifier, cryptographic hash values and the executing user account.

data.win.eventdata.timestamp ~ data.win.eventdata.timestamp										
timestamp	agent.name	rule.desc...	r...	rul...	data.win.eventdata.image	data.win.eventdata.command...	data.win.eventdata.parentima...	da...	data.win.eventdata.hashes	data.win.eventdata.user
Jul 15, 2026 @ 13:51:38.078	DESKTOP-P1FP8HG	Powershell...	12	92057	C:\\Windows\\System32\\Window...	!C:\\WINDOWS\\System32\\Wind...	C:\\Windows\\System32\\Window...	1896	SHA256=0FF6F2C94BC7E2833A...	DESKTOP-P1FP8HG\\C\\SA303-Wi...

Field	Value
Image	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
Command Line	<pre> "C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe" -NoProfile -EncodedCommand IgbDAFMAQQAzADAAMwAgAGUAbgBjAG8AZABlAGQAI ABQAG8AdwBlAHIAUwBoAGUABsACAAAdABlAHMAdA AiACAAfAAgAFMAZQB0AC0AQwBvAG4AdABlAG4AdAA gAEMA0gBcAEMAUwBBADMAMAAzAC0ARwBlAG4AZQBy AGEAdABlAGQAXABlAG4AYwBvAGQAZQBkAC0AYwBvA G0AbQBhAG4AZAAuAHQAeAB0AA== </pre>
Parent Image	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
User	DESKTOP-P1FP8HG\CSA303-WIN-ASA1564
Process ID	1896
SHA256 Hash	SHA256=0FF6F2C94BC7E2833A5F7E16DE1622E5DBA70396F31C7D5F56381870317E8C46, IMPHASH=AFACF6DC9041114B198160AAB4D0AE77
UTC Timestamp	Jul 15, 2026 @ 13:51:38.078

Investigation of Certutil Activity

The investigation identified Certutil execution within the same observation window. Examination of the recorded command-line arguments confirmed that the utility was used exclusively to encode and decode local files contained within the C:\CSA303-Generated directory.

Jul 15, 2026 @ 13:32:24.162 - Jul 15, 2026 @ 16:32:24.162					
Export Formatted Reset view 735 available fields Columns Density Sort fields Full screen					
agent.name	rule.level	data.win.eventdata.image	data.win.eventdata.commandLine	rule.id	
DESKTOP-P1FP8HG	6	C:\\Windows\\System32\\certutil.exe	"C:\\WINDOWS\\system32\\certutil.exe" ...	92073	

The following command was observed:

"C:\\Windows\\System32\\certutil.exe" -decode

C:\\CSA303-Generated\\plain.b64

C:\\CSA303-Generated\\decoded.txt

Reconstruct the ordered sequence

Order	Event	Evidence
1	Encoded file plain.b64 existed before decoding.	Inferred because certutil cannot decode a file that does not already exist.
2	certutil.exe decoded plain.b64.	Command line shows -decode.
3	decoded.txt was created	Output file specified in the command line

Investigation of Windows Scheduled Task Activity

During the laboratory exercise, the PowerShell script intentionally created a temporary scheduled task named CSA303-Benign-Task. The purpose of the task was to simulate attacker behaviour in a controlled manner while ensuring that no permanent persistence mechanism remained on the endpoint after execution. The investigation focused on determining whether the scheduled task was successfully created, executed and subsequently removed.

The first relevant event showed the creation of the scheduled task using the /Create parameter. Examination of the recorded command line confirmed that the task was configured to execute a simple Command Prompt instruction that generated a text file within the laboratory directory.

```
TimeCreated : 7/15/2026 1:51:02 PM
Id          : 1
Message     : Process Create:
              RuleName: -
              UtcTime: 2026-07-15 12:51:02.244
              ProcessGuid: {73557c48-8236-6a57-0f04-00000000b00}
              ProcessId: 7632
              Image: C:\Windows\System32\schtasks.exe
              FileVersion: 10.0.26100.4343 (WinBuild.160101.0800)
              Description: Task Scheduler Configuration Tool
              Product: Microsoft® Windows® Operating System
              Company: Microsoft Corporation
              OriginalFileName: schtasks.exe
              CommandLine: "C:\WINDOWS\system32\schtasks.exe" /Run /TN CSA303-Benign-Task
              CurrentDirectory: C:\Users\CSA303-WIN-ASA15643\Desktop\CSA303LAB1\
              User: DESKTOP-P1FP8HG\CSA303-WIN-ASA15643
              LogonGuid: {73557c48-5927-6a57-7adc-020000000000}
              LogonId: 0x2DC7A
              TerminalSessionId: 1
              IntegrityLevel: High
              Hashes: SHA256=2D81DA2A099A2F25CD9C0880BC2AC9271ACEA162CA2AD8E55A554D86DD81D36C, IMPHASH=3442ADA9D4D3
                    7ECAE031208025BC1024
              ParentProcessGuid: {73557c48-7e26-6a57-cd03-00000000b00}
              ParentProcessId: 9724
              ParentImage: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
              ParentCommandLine: "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"
              ParentUser: DESKTOP-P1FP8HG\CSA303-WIN-ASA15643
```

"C:\WINDOWS\system32\schtasks.exe" /Create /TN CSA303-Benign-Task /TR "cmd.exe /c echo CSA303 scheduled task test for YOUR-STUDENT-ID > C:\CSA303-Generated\task-output.txt" /SC ONCE /ST 23:59 /F

The second Sysmon event recorded the execution of the newly created scheduled task. The command line contained the /Run parameter, indicating that the task was manually triggered immediately after creation rather than waiting for its scheduled execution time.

```

TimeCreated : 7/15/2026 1:51:02 PM
Id          : 1
Message     : Process Create:
              RuleName: -
              UtcTime: 2026-07-15 12:51:02.244
              ProcessGuid: {73557c48-8236-6a57-0f04-00000000b00}
              ProcessId: 7632
              Image: C:\Windows\System32\schtasks.exe
              FileVersion: 10.0.26100.4343 (WinBuild.160101.0800)
              Description: Task Scheduler Configuration Tool
              Product: Microsoft® Windows® Operating System
              Company: Microsoft Corporation
              OriginalFileName: schtasks.exe
              CommandLine: "C:\WINDOWS\system32\schtasks.exe" /Run /TN CSA303-Benign-Task
              CurrentDirectory: C:\Users\CSA303-WIN-ASA15643\Desktop\CSA303LAB1\
              User: DESKTOP-P1FP8HG\CSA303-WIN-ASA15643
              LogonGuid: {73557c48-5927-6a57-7adc-020000000000}
              LogonId: 0x2DC7A
              TerminalSessionId: 1
              IntegrityLevel: High
              Hashes: SHA256=2D81DA2A099A2F25CD9C0880BC2AC9271ACEA162CA2AD8E55A554D86DD81D36C, IMPHASH=3442ADA9D4D3
                    7ECAE031208025BC1024
              ParentProcessGuid: {73557c48-7e26-6a57-cd03-00000000b00}
              ParentProcessId: 9724
              ParentImage: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
              ParentCommandLine: "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"
              ParentUser: DESKTOP-P1FP8HG\CSA303-WIN-ASA15643

```

"C:\WINDOWS\system32\schtasks.exe" /Run /TN CSA303-Benign-Task

Following successful execution, the PowerShell script removed the scheduled task using the /Delete parameter.

```

7/15/2026 1:51:05 PM
1
Process Create:
  RuleName: -
  UtcTime: 2026-07-15 12:51:05.639
  ProcessGuid: {73557c48-8239-6a57-1004-00000000b00}
  ProcessId: 3492
  Image: C:\Windows\System32\schtasks.exe
  FileVersion: 10.0.26100.4343 (WinBuild.160101.0800)
  Description: Task Scheduler Configuration Tool
  Product: Microsoft® Windows® Operating System
  Company: Microsoft Corporation
  OriginalFileName: schtasks.exe
  CommandLine: "C:\WINDOWS\system32\schtasks.exe" /Delete /TN CSA303-Benign-Task /F
  CurrentDirectory: C:\Users\CSA303-WIN-ASA15643\Desktop\CSA303LAB1\
  User: DESKTOP-P1FP8HG\CSA303-WIN-ASA15643
  LogonGuid: {73557c48-5927-6a57-7adc-020000000000}
  LogonId: 0x2DC7A
  TerminalSessionId: 1
  IntegrityLevel: High
  Hashes: SHA256=2D81DA2A099A2F25CD9C0880BC2AC9271ACEA162CA2AD8E55A554D86DD81D36C, IMPHASH=3442ADA9D4D3
        7ECAE031208025BC1024
  ParentProcessGuid: {73557c48-7e26-6a57-cd03-00000000b00}
  ParentProcessId: 9724
  ParentImage: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
  ParentCommandLine: "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"
  ParentUser: DESKTOP-P1FP8HG\CSA303-WIN-ASA15643

```

"C:\WINDOWS\system32\schtasks.exe" /Delete /TN CSA303-Benign-Task /F

During the investigation, the expected schtasks.exe events could not initially be located in the Wazuh Dashboard. To verify whether the events had been generated, the Sysmon Operational log on the Windows endpoint was queried directly using PowerShell instead of relying solely on the SIEM.

The following command was executed:

```
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" |
```

```
Where-Object {$_.Message -match "schtasks"} |
```

```
Select-Object TimeCreated, Id, Message -First 5
```

The command successfully returned the Sysmon Process Creation (Event ID 1) and Process Termination (Event ID 5) events associated with schtasks.exe. These logs confirmed that the scheduled task was successfully created, executed, and deleted. The evidence obtained from the endpoint was therefore used to complete the investigation, demonstrating the importance of validating security events directly from the source when they are not immediately available in the Wazuh Dashboard.

Event Correlation and Timeline Reconstruction

Using the UTC timestamps generated by the laboratory script together with the Sysmon Process Creation events, the execution sequence was reconstructed. The resulting timeline demonstrated that the generated activities followed a logical progression beginning with PowerShell execution and concluding with removal of the temporary scheduled task.

UTC Time	Source / Event ID	Entity	Observed Fact	Analytical Interpretation
2026-07-15 12:51:02	Sysmon Operational – Event ID 5	schtasks.exe	The command "C:\WINDOWS\system32\schtasks.exe" /Create /TN CSA303-Benign-Task /TR "cmd.exe /c echo CSA303 scheduled task test..." created the scheduled task CSA303-Benig	The task was intentionally created by the laboratory script to simulate Scheduled Task activity. In a real environment, similar behaviour may indicate persistence (MITRE ATT&CK T1053)

			n-Task.	
2026-07-15 12:51:02	Sysmon Operational – Event ID 5	schtasks.exe	The command "schtasks.exe" /Run /TN CSA303-Benign-Task executed the newly created task.	Immediate execution confirmed that the scheduled task successfully launched its configured action rather than remaining dormant.
2026-07-15 12:51:05	Sysmon Operational – Event ID 5	schtasks.exe	The command "schtasks.exe" /Delete /TN CSA303-Benign-Task /F permanently removed the scheduled task.	Deleting the task immediately after execution indicates that it was temporary. Attackers sometimes delete artefacts to reduce forensic evidence, although in this case it was expected laboratory behaviour
2026-07-15 12:51:05	Sysmon Operational – Event ID 5	schtasks.exe	Sysmon recorded the termination of schtasks.exe after completing the	Confirms that the Scheduled Task utility completed execution successfully and exited normally.

			delete operation.	
2026-07-15 12:50:59.116	Sysmon Operational – Event ID 1	powershell.exe	PowerShell executed using the -EncodedCommand parameter with Base64-encoded instructions to create C:\CSA303-Generated\encoded-command.txt.	Encoded PowerShell commands are commonly associated with attacker obfuscation techniques; however, the command content and destination confirm this was a controlled laboratory simulation rather than malicious execution.
2026-07-15 12:51:02.063	Sysmon Operational – Event ID 1	certutil.exe	CertUtil executed the command certutil.exe -decode C:\CSA303-Generated\plain.b64 C:\CSA303-Generated\decoded.txt.	CertUtil successfully decoded a local Base64 file. Although CertUtil is frequently abused by attackers, the local file paths and absence of network activity indicate legitimate testing activity.

Distinguishing Fact from Inference

Throughout the investigation, care was taken to distinguish directly observed evidence from conclusions drawn through analytical reasoning.

Fact	Inference
PowerShell executed using the -EncodedCommand parameter.	The encoded command formed part of the authorised laboratory exercise.
CertUtil decoded a Base64-encoded local file.	No malicious payload was introduced because all files remained within the laboratory directory.
The scheduled task was created, executed and deleted	The task represented temporary automation rather than persistence.
All parent processes originated from PowerShell.	The execution chain was consistent with the supplied laboratory script.

Legitimate Administrative Activities Producing Similar Telemetry

The investigation demonstrated that several legitimate administrative tasks generate telemetry nearly identical to behaviour commonly associated with cyber-attacks. Consequently, analysts should avoid escalating alerts solely because a particular executable or command-line argument appears suspicious.

For example, system administrators frequently use PowerShell to automate software deployment, system maintenance and configuration management. Likewise, CertUtil may legitimately be used to encode or decode certificate-related files, while Scheduled Tasks are routinely employed to automate backups, software updates and scheduled maintenance activities.

Before escalating similar activity as a potential security incident, analysts should consider additional contextual evidence, including:

- Whether the activity occurred during an approved maintenance window.
- Whether the executing user account was authorised to perform administrative tasks.
- Whether the parent process was expected or unusual.
- Whether external network communication occurred.
- Whether the observed behaviour aligns with documented organisational procedures.

Considering these contextual factors reduces false positives while improving the accuracy of threat detection and incident response activities.

CREATION AND VALIDATION OF CUSTOM WAZUH RULES

The first custom rule was developed to identify PowerShell processes executed with encoded commands. Instead of searching for a specific Base64 string, the rule examines the executable image together with the command-line arguments recorded by Sysmon. This approach enables the rule to detect any encoded PowerShell execution regardless of the encoded content itself.

The rule applies only to Sysmon Process Creation events (Event ID 1), ensuring that the detection focuses on process execution rather than unrelated Windows events.

Field	Value
parent group	sysmon_event1
field	win.eventdata.image
Regular expression	(?i)\\powershell(?:\\.exe)?\$

Severity	12
Description	CSA-303: PowerShell launched with encoded-command indicator

Show one alert for PowerShell and one for Certutil

*Powershell.exe Rule ID-92057, Level-6, Agent-DESKTOP-PIFP8HG, Matched
Field-win.eventdata.commandLine containing -EncodedCommand*

1 hit					
Jul 15, 2026 @ 13:41:23.996 - Jul 15, 2026 @ 16:41:23.996					
 Export Formatted  Reset view  735 available fields  Columns  Density  Sort fields  Full screen					
	agent.name	rule.level	data.win.eventdata.image	data.win.eventdata.commandLine	rule.id
	DESKTOP-P1FP8HG	12	C:\\Windows\\System32\\WindowsP...	\"C:\\WINDOWS\\System32\\WindowsPow...	92057

*certutil.exe Rule ID-92073, Level-6, Agent-DESKTOP-PIFP8HG, Matched Field-win.eventdata.commandLine
containing -decode (or -encode)*

1 hit					
Jul 15, 2026 @ 13:32:24.162 - Jul 15, 2026 @ 16:32:24.162					
 Export Formatted  Reset view  735 available fields  Columns  Density  Sort fields  Full screen					
	agent.name	rule.level	data.win.eventdata.image	data.win.eventdata.commandLine	rule.id
	DESKTOP-P1FP8HG	6	C:\\Windows\\System32\\certutil.exe	\"C:\\WINDOWS\\system32\\certutil.exe\" ...	92073

Proposed Detection Tuning

Although both custom rules successfully detected the laboratory activities, deploying them in a production environment without refinement could generate unnecessary alerts during routine administrative operations.

One defensible tuning recommendation would be to limit the encoded PowerShell rule to approved administrative service accounts or authorised management servers.

Owner: SOC Team / Windows Infrastructure Team

Evidence: Routine software deployment uses encoded PowerShell from Microsoft Endpoint Configuration Manager (MECM) servers.

Tuning: Exclude approved management servers only after verification.

Expiry Requirement: Review every six months or immediately after infrastructure changes.

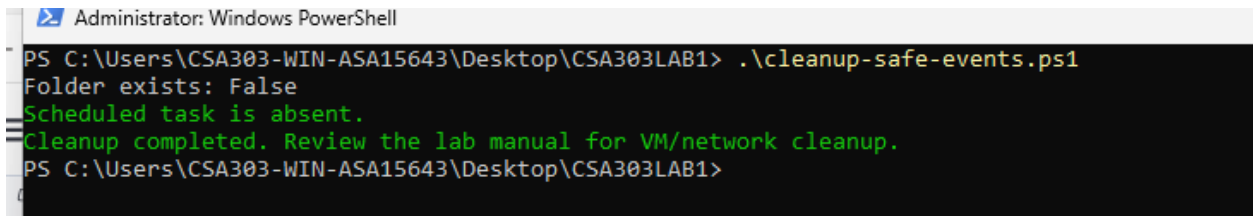
This tuning approach maintains visibility into potentially suspicious encoded PowerShell activity while reducing alerts generated by known and authorised administrative processes. Importantly, the recommendation avoids implementing broad exclusions, ensuring that encoded PowerShell execution originating from user workstations or unknown systems continues to generate alerts for investigation.

CLEANUP AND RESTORATION

The supplied cleanup script was executed with administrative privileges to automatically remove all files and artefacts created during the safe event generation phase.

The following command was executed:

`.\cleanup-safe-events.ps1`



```
Administrator: Windows PowerShell
PS C:\Users\CSA303-WIN-ASA15643\Desktop\CSA303LAB1> .\cleanup-safe-events.ps1
Folder exists: False
Scheduled task is absent.
Cleanup completed. Review the lab manual for VM/network cleanup.
PS C:\Users\CSA303-WIN-ASA15643\Desktop\CSA303LAB1>
```

After cleaning up verification also was done

```
PS C:\Users\CSA303-WIN-ASA15643\Desktop\CSA303LAB1> schtasks /Query /TN "CSA303-Benign-Task"
ERROR: The system cannot find the file specified.
PS C:\Users\CSA303-WIN-ASA15643\Desktop\CSA303LAB1> _
Cleanup completed. Review the lab manual for VM/network cleanup.
PS C:\Users\CSA303-WIN-ASA15643\Desktop\CSA303LAB1> Test-Path C:\CSA303-Generated
False
PS C:\Users\CSA303-WIN-ASA15643\Desktop\CSA303LAB1>
```

As expected it return false and error for the second one

Confirming the removal of scheduled tasks is particularly important because Scheduled Tasks are widely used by attackers as persistence mechanisms. Although the laboratory task was intentionally benign, verifying its deletion demonstrates that no persistent automation remained on the endpoint after testing.

Analysis and Findings

The laboratory demonstrated how Sysmon and the Wazuh SIEM can be used to collect, monitor and analyse Windows endpoint telemetry. Security events generated on the endpoint were successfully forwarded to the Wazuh Manager, enabling the analysis of process execution, parent-child relationships and command-line activity.

Sysmon provided detailed process information, including executable paths, command-line arguments, process IDs, parent processes and hash values, which improved the ability to reconstruct event sequences during the investigation. Analysis confirmed that the encoded PowerShell command, although commonly associated with malicious activity, originated from the authorised laboratory script and showed no evidence of network communication or malicious behaviour.

Similarly, the CertUtil activity demonstrated that legitimate Windows utilities can generate telemetry similar to attacker techniques. The decoded file remained within the local laboratory directory, reinforcing the importance of analysing behavioural context rather than relying solely on executable names.

Correlation of the Scheduled Task events showed that the task was created, executed and deleted within a short period, confirming that it was a temporary laboratory activity rather than a persistence mechanism. Additionally, the custom Wazuh rules successfully detected both the encoded PowerShell and CertUtil events, demonstrating that command-line-based detections provide more effective and accurate monitoring than executable-name matching alone.

Overall, the investigation highlighted the importance of endpoint telemetry, event correlation and contextual analysis in enabling SOC analysts to distinguish legitimate administrative activities from potential security threats.

CHALLENGES ENCOUNTERED

Several challenges were encountered during the completion of this laboratory exercise. Addressing these issues provided valuable practical experience in troubleshooting endpoint monitoring systems and reinforced the importance of validating collected telemetry before beginning an investigation.

The first challenge occurred during execution of the PowerShell event generation script. Initially, PowerShell reported that the script could not be found because commands located in the current working directory are not executed automatically. This issue was resolved by executing the script using the relative path notation (`.\generate-safe-events.ps1`), which explicitly instructed PowerShell to execute the script located in the current directory.

A second challenge involved the PowerShell execution policy, which prevented unsigned scripts from executing. This was resolved by temporarily changing the execution policy for the current PowerShell process using the `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` command. Restricting the policy modification to the current session ensured that the system's permanent security configuration remained unchanged.

Another significant challenge occurred during the investigation phase when several expected Scheduled Task events could not initially be located within the Wazuh Dashboard. Although Sysmon had successfully recorded the events, they were not immediately searchable through Wazuh. Further investigation using Windows Event Viewer confirmed that the events existed within the Sysmon Operational log, indicating that the issue was related to event indexing or forwarding rather than event generation. This experience highlighted the importance of validating endpoint logs using multiple evidence sources rather than relying solely on a single monitoring platform.

Finally, interpreting parent-child process relationships required careful analysis during the investigation. Initially, identifying child processes based solely on executable names proved confusing. However, by examining the ParentImage and Image fields together, it became possible to reconstruct the execution chain accurately and understand how PowerShell launched CertUtil and Scheduled Task operations during the laboratory exercise.

These challenges reinforced the importance of systematic troubleshooting, evidence validation and analytical reasoning throughout the incident investigation process.

Recommendations

Based on the findings from this laboratory, the following recommendations are proposed:

- **Deploy an optimized Sysmon configuration** (e.g., SwiftOnSecurity or Olaf Hartong) to improve endpoint visibility while reducing unnecessary event noise.
- **Prioritize behavioural detection rules** by monitoring command-line arguments, parent-child relationships and process behaviour instead of relying only on executable names.

- **Continue monitoring encoded PowerShell execution**, as it is commonly associated with attacker activity. Legitimate administrative use should be validated using user, host and execution context.
- **Monitor Living-off-the-Land Binaries (LOLBins)** such as PowerShell, CertUtil, WMI, Rundll32 and Mshta, as these trusted tools are frequently abused by attackers.
- **Regularly validate and tune Wazuh custom rules** using controlled testing (e.g., wazuh-logtest) to ensure detections remain effective after updates or configuration changes.
- **Maintain baseline reviews of endpoint activity** to help SOC analysts distinguish normal system behaviour from suspicious or anomalous events, reducing false positives and improving investigation efficiency.

CONCLUSION

This laboratory successfully demonstrated the practical application of endpoint monitoring, threat detection and incident investigation using Sysmon and the Wazuh SIEM platform. Beginning with the deployment and validation of endpoint monitoring components, the exercise progressed through baseline establishment, controlled event generation, threat investigation, event correlation, custom detection rule validation and secure environment restoration.

The investigation highlighted the importance of analysing endpoint telemetry within its operational context. Although encoded PowerShell commands, CertUtil operations and Scheduled Tasks are commonly associated with malicious activity, the laboratory demonstrated that identical telemetry can also result from legitimate administrative processes. Consequently, effective threat hunting requires analysts to examine command-line arguments, parent-child process relationships, user accounts, timestamps and surrounding behavioural evidence before reaching investigative conclusions.

The laboratory also reinforced the value of behavioural detection within modern SIEM platforms. Custom Wazuh rules successfully identified encoded PowerShell execution and CertUtil activity based on command-line analysis, illustrating how organisations can extend built-in detections to improve visibility into techniques commonly abused by threat actors.

Overall, the exercise strengthened practical skills in endpoint monitoring, forensic investigation, event correlation, detection engineering and security reporting. These competencies form essential components of modern Security Operations Centre (SOC) workflows and provide a solid foundation for future threat hunting and incident response activities.